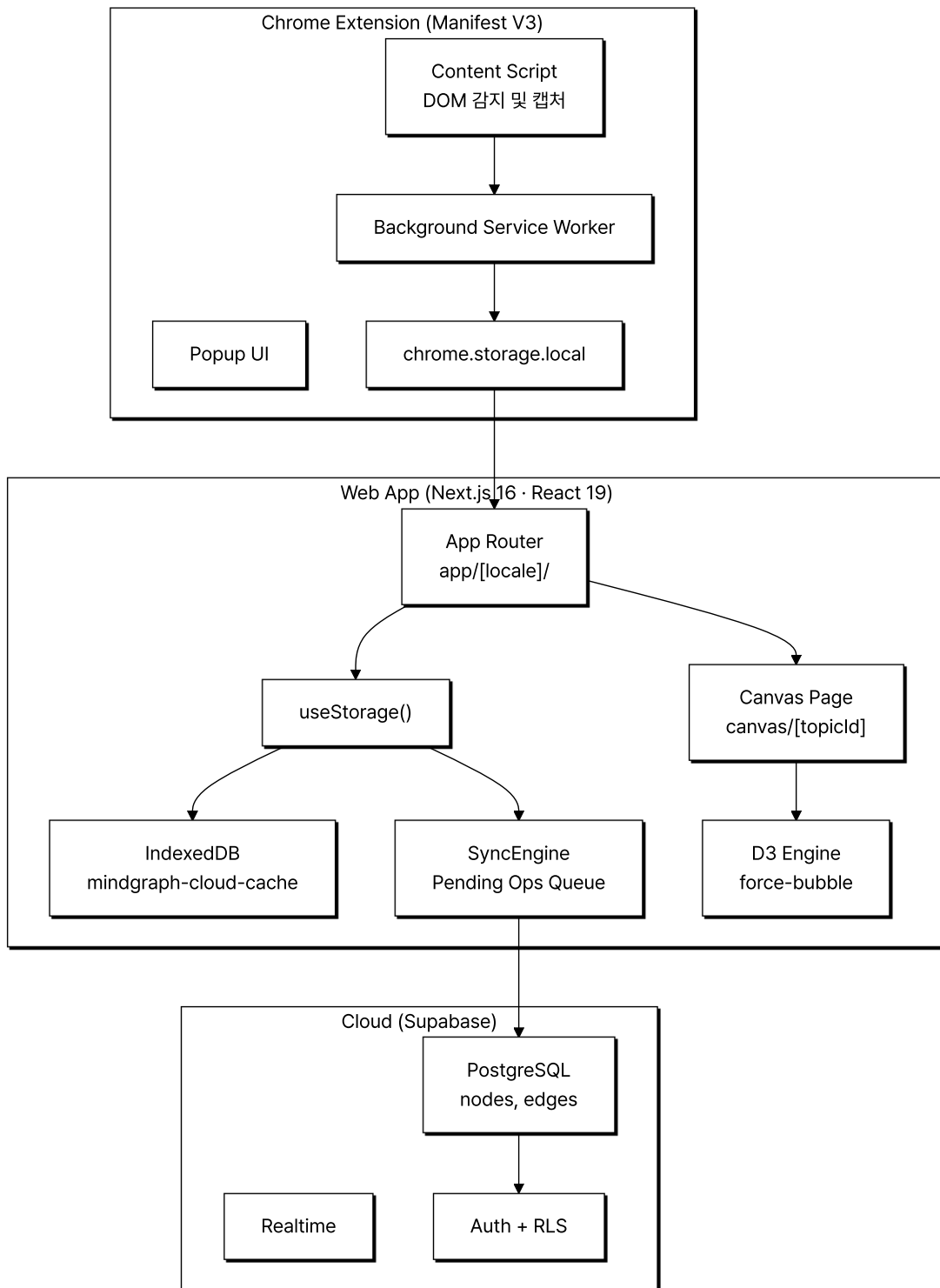


Portfolio

[MINDGRAPH] - AI 지식 캡처 & 그래프 시각화

ChatGPT·Gemini·Claude·Grok 4개 LLM 서비스의 답변을 캡처해 의미 단위로 묶고 D3.js로 시각화하는 Chrome Extension·Next.js 웹앱입니다. "여러 LLM 서비스에 흩어진 답변을 한 곳에 모으면 사용자의 사고 흐름이 보존된다"는 가설을 코드보다 문제 정의에서 먼저 잡고, 1인 PO가 Chrome Extension·Next.js 웹앱·Supabase 백엔드를 풀스택으로 책임지며 도메인 getmindgraph.com 출시 대비 1인 프로토타입 빌드를 주도적 오너십으로 진행하고 있습니다. 가설을 빠르게 검증하기 위해 Claude Code 위에 9개 혹·plan·qa·ship 3 phase /sprint 워크플로우·4중 SSOT 자동 동기를 직접 설계했습니다 (Case 1에서 상세히 설명합니다).

전체적인 아키텍처



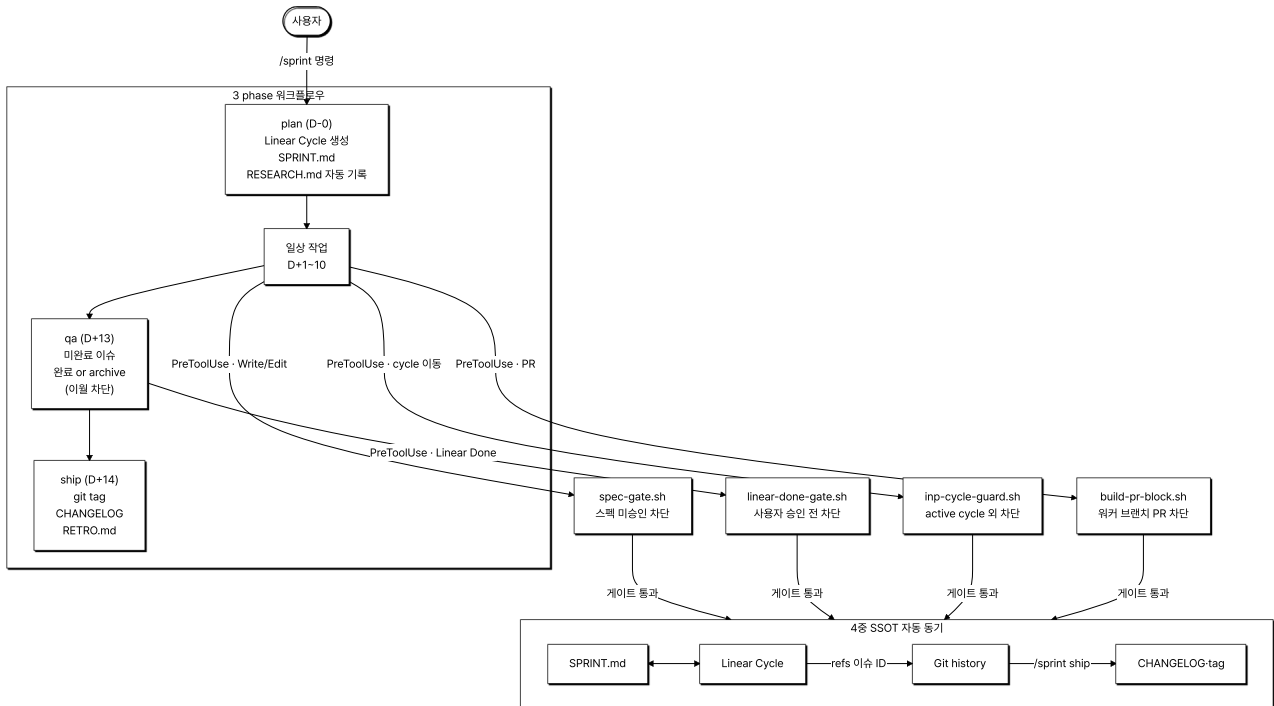
- **Architecture:** Chrome Extension(MV3)이 캡처 전용 도구, Next.js 16 App Router 웹앱이 지식 정리·그래프 편집·검색·동기화 허브. Extension 빌드는 `vite.config.extension.ts` · `vite.config.content.ts` · `vite.config.bridge.ts` 3개 vite 설정으로 Service Worker·Content Script·인증 bridge 번들을 분리했습니다.

Case 1. 1인 PO 환경에서 AI를 팀처럼 운영하는 워크플로우 시스템 설계

1. 문제 원인

- 1인 PO가 기획·구현·QA·배포를 혼자 담당하면서 AI와의 대화가 세션 간 초기화되어 같은 규칙·교정을 반복 설명하는 비용이 다음 가설 검증 시간을 잡아먹었습니다.
- AI가 사용자 확인 없이 commit 생성·이슈 완료 처리 같은 자율 행동을 해서 검증 안 된 변경이 그대로 main 브랜치에 머지되는 사고 위험이 있었습니다.
- 코드·이슈·문서·배포 기록 네 시스템이 같은 정보를 중복 관리해 현재 상태 파악을 위해 네 곳을 수동 대조해야 했습니다.

2. 해결 과정



- **9개 혹은 자동 개입:** 도구 호출 시점에 자동 실행되는 9개 혹은 두어 차단 게이트 4개(**spec-gate.sh** , **linear-done-gate.sh** , **build-pr-block.sh** , **inp-cycle-guard.sh**)와 보조 알림 5개(**stale-warn.sh** , **inp-reuse-suggest.sh** , **integrity-sync.sh** , **context-pack-stale.sh** , **worktree-env-symlink.sh**)로 분류했습니다.
- **/sprint 메타 스킬 3 phase:** plan(Linear Cycle 생성·SPRINT.md 작성·직전 cycle RESEARCH.md 자동 기록)·qa(미완료 이슈를 완료 또는 archive로 강제 결정, 이월 옵션 mechanism 차단)·ship(git tag·CHANGELOG·RETRO.md 자동 기록)을 하나의 명령으로 연결하고, 각 phase 진입 전 이전 phase 산출물 존재 여부를 자동 검증했습니다. 옛 6단계(research/build/retro)는 plan의 RESEARCH.md 자동 기록과 ship의 RETRO.md 자동 기록으로 흡수했습니다.
- **4중 SSOT:** SPRINT.md(문서)·Linear Cycle(이슈)·Git(코드)·CHANGELOG(배포 기록)에 역할을 하나씩만 부여하고 **/sprint** 가 단계마다 자동 동기화했습니다.
- **5층 컨텍스트 영속화:** Conversation-Memory·CLAUDE.md·Hook-Skill 5층에 규칙을 분산해 교정이 바로 반영되고 다음 세션에 자동 로드되며 매 세션 system prompt에 강제되고 도구 호출 시 차단됩니다.
- **부서·에이전트 분리 + context-map 자동 라우팅:** 7개 부서(design·dev·docs·marketing·ops·product·qa)와 9개 에이전트(ceo·frontend/backend·qa-engineer·product-manager·ui-ux-designer·marketing-strategist·ops-engineer·knowledge-logger)별로 **department/{팀}/CLAUDE.md** 와 **docs/ lifecycle** 폴더를 분리하고, **RULES/context-map.md** + **SYSTEM/schemas/code-doc-mapping.yaml** 로 task_type별 분류(new_feature·ui_change·api_change·bug_fix·refactor 등) 과 코드 path glob을 must-read doc에 매핑했습니다. CEO 에이전트가 워커에 위임할 때 합집합 5개 내외 doc만 prompt에 첨부되어 매 세션 LLM이 받는 컨텍스트 표면이 좁혀졌고, PO가 가설을 코드로 옮길 때 LLM 응답 정확도와 토큰 비용을 동시에 잡았습니다.

3. 결과

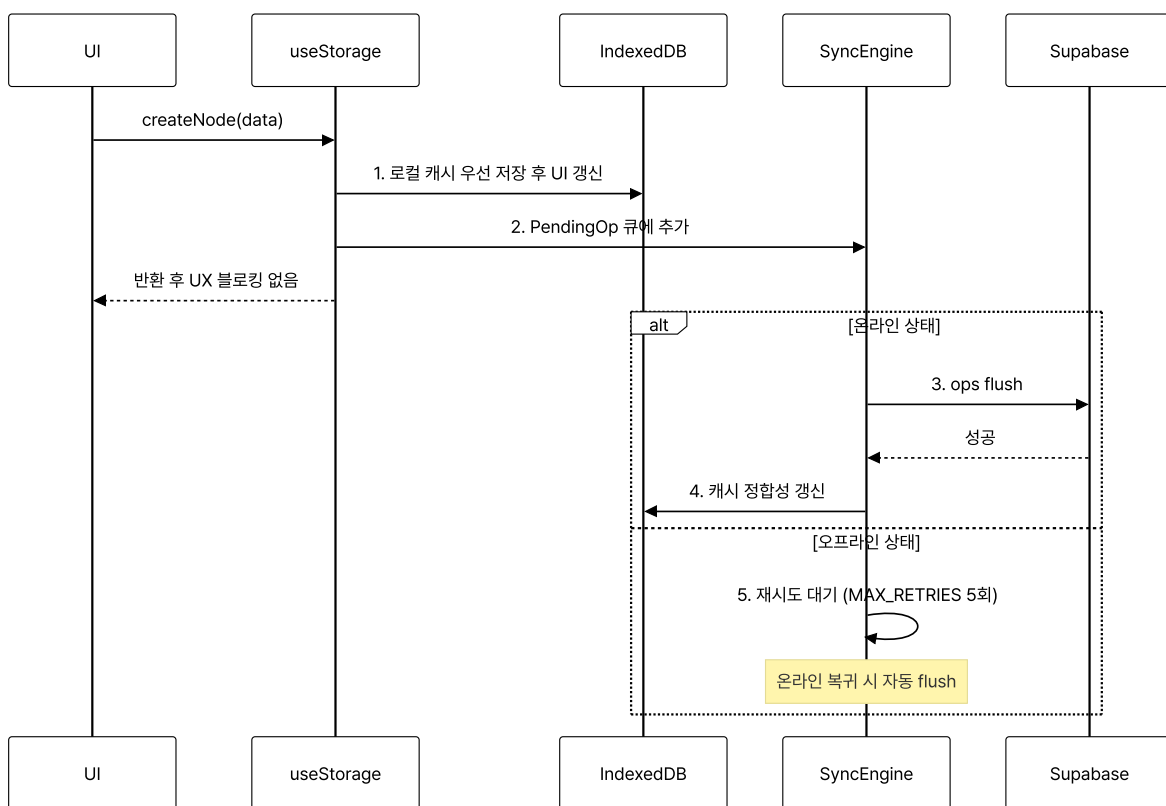
- **성과:** 이슈 등록·SPRINT.md 작성·CHANGELOG 갱신·Git 태그·Linear Done 전환 같은 sprint 라이프사이클 반복 작업을 9개 혹은 `/sprint` 3 phase로 자동화해 PO가 가설 검증에 시간을 다시 쓸 수 있게 했습니다.
- **배운 점:** 9개 혹은 3 phase `/sprint`, 4종 SSOT 자동 동기를 코드화한 결과 가설 검증에 쓰는 시간 비중이 다시 늘었습니다.

Case 2. 끊임 없는 캡처 UX를 위한 Write-Behind 캐시 설계

1. 문제 원인

- 캡처는 카페·지하철 같은 이동 중에 가장 자주 일어나리라 가정했는데 초기 구조가 Supabase 직접 쓰기여서 네트워크 단절 시 사용자 입력이 사라질 수 있는 구조였고, PO 입장에서 가장 큰 사용자 이탈 요인이 될 것이라 판단했습니다.
- 온라인 복귀 후 누락 노드를 수동 재입력해야 하면 새 캡처 흐름의 신뢰도가 떨어져 출시 후 다음 가설 검증을 시작하지 못한다는 리스크가 있다고 봤습니다.

2. 해결 과정



- **읽기 경로:** `useStorage()` 혹은 항상 IndexedDB 캐시에서 먼저 응답해 네트워크와 무관하게 UI가 갱신되도록 했습니다.
- **쓰기 경로:** 로컬 캐시에 먼저 쓴 뒤 `syncToCloud()` 가 `addPendingOps` 로 큐에 기록하고 `flushPendingOps` 를 await 없이 호출하는 Write-Behind 방식을 적용했습니다.
- **재시도 정책:** `sync-engine.ts` 의 `MAX_RETRIES = 5` 상수로 실패한 op의 재시도 한도를 두고, 초과 시 `status: 'failed'` 로 마킹해 재시도 비용 폭발을 차단했습니다.
- **Realtime 통합:** Supabase Realtime 채널 수신 시 `use-sync-manager` 가 `refreshCache()` 를 호출해 멀티 디바이스 동기화를 처리했습니다.
- **캐시 분리:** 클라우드 캐시(`mindgraph-cloud-cache`)와 로컬 데이터(`mindgraph-web`)를 별도 IndexedDB로 두어 로그아웃 시 클라우드만 비우고 비로그인 로컬 데이터는 유지했습니다.

3. 결과

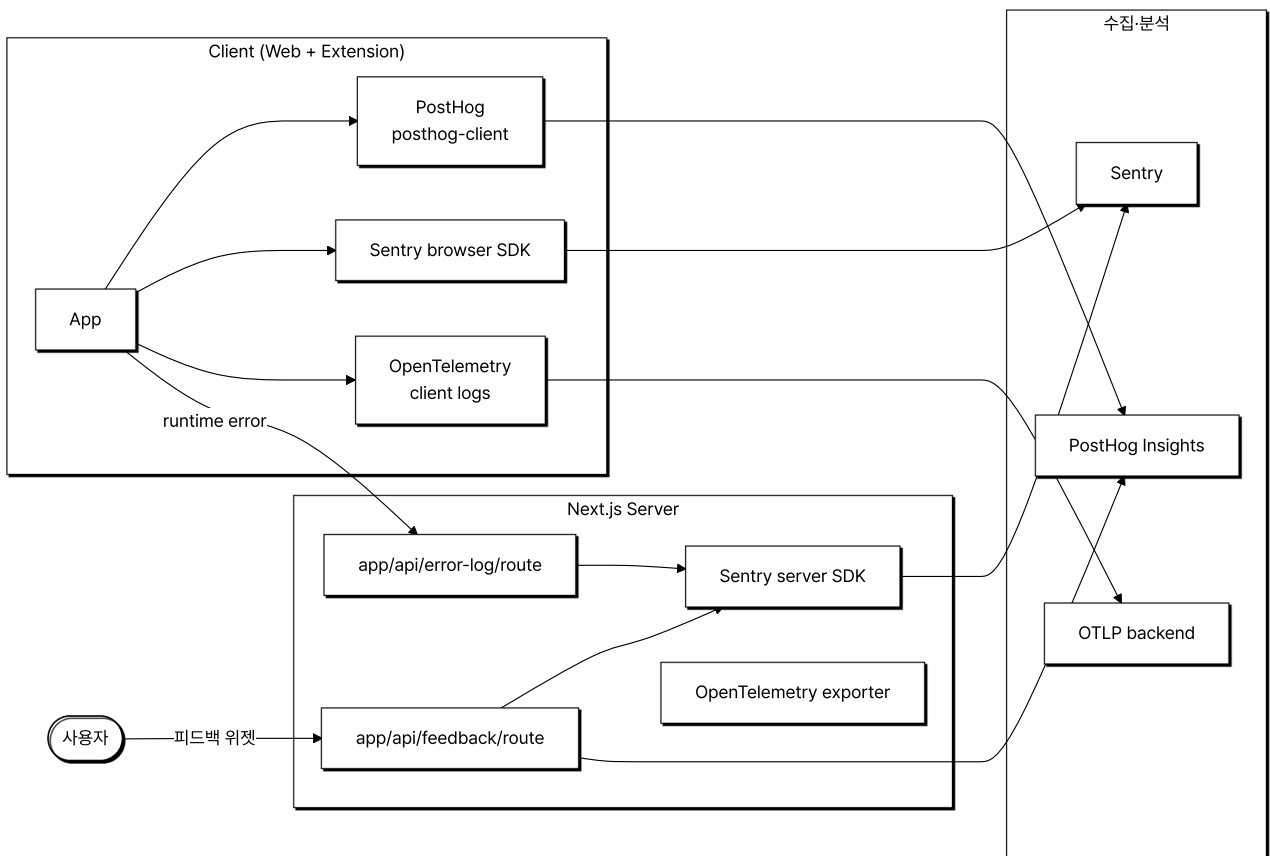
- **성과:** 오프라인 상태에서 노드 CRUD가 동작하고 온라인 복귀 시 Pending Ops 큐가 자동 flush되며 멀티 디바이스 동기화 구조를 확보했습니다.
- **배운 점:** "끊김 없는 캡처"라는 PO 약속을 IndexedDB 분리·pending 큐·Realtime 통합·재시도 정책을 묶은 시스템으로 풀어, 오프라인에서도 캡처가 유실되지 않게 했습니다.

Case 3. 관측성·피드백 인프라 사전 구축으로 출시 후 가설 검증 사이클 짧게 닫는 설계

1. 문제 원인

- 1인 PO 환경에서 출시 후 사용자가 어디서 막힐지 받을 데이터 채널이 없으면 다음 가설 결정 근거가 부재할 것이라 판단했습니다.
- 사용자 에러·요청을 받을 통로를 출시 전에 만들어두지 않으면 출시 후 발생할 사용자 사고가 다음 sprint plan에 반영될 통로 없이 누락된다고 판단했습니다.
- 클라이언트 에러와 서버 에러, 사용자 행동 이벤트가 분리된 시스템에 흩어져 있으면 출시 후 동일 사용자의 흐름을 한 줄로 추적하기 어려운 구조가 되리라 봤습니다.

2. 해결 과정



- **PostHog 행동 이벤트:** `posthog-client.ts` · `posthog-provider.tsx` 로 사용자 행동(캡처·노드 클릭·검색)을 이벤트로 적재할 funnel·전환율 측정 기반을 사전 구축했습니다.
- **Sentry 에러 수집:** `sentry.edge.config.ts` · `sentry.server.config.ts` · browser SDK로 클라이언트·서버·edge 런타임 에러를 한곳에 모을 채널을 코드 베이스에 함께 두었습니다.
- **사용자 피드백 라우트:** `app/api/feedback/route.ts` 로 사용자 피드백을 받을 라우트를 사전 구축하고 Sentry에 연동해, 출시 후 피드백을 다음 sprint plan 입력으로 곧장 이어 붙일 수 있도록 했습니다.
- **에러 로그 라우트:** `app/api/error-log/route.ts` 로 클라이언트가 잡지 못한 에러를 서버 측에 기록할 통로를 사전 마련해 출시 후 누락을 차단했습니다.

- **OpenTelemetry 로그:** `instrumentation.ts` · `instrumentation-client.ts` 에 OpenTelemetry SDK를 두어 서버·클라이언트 로그를 OTLP exporter로 통합 수집할 동선을 사전 설계했습니다.

3. 결과

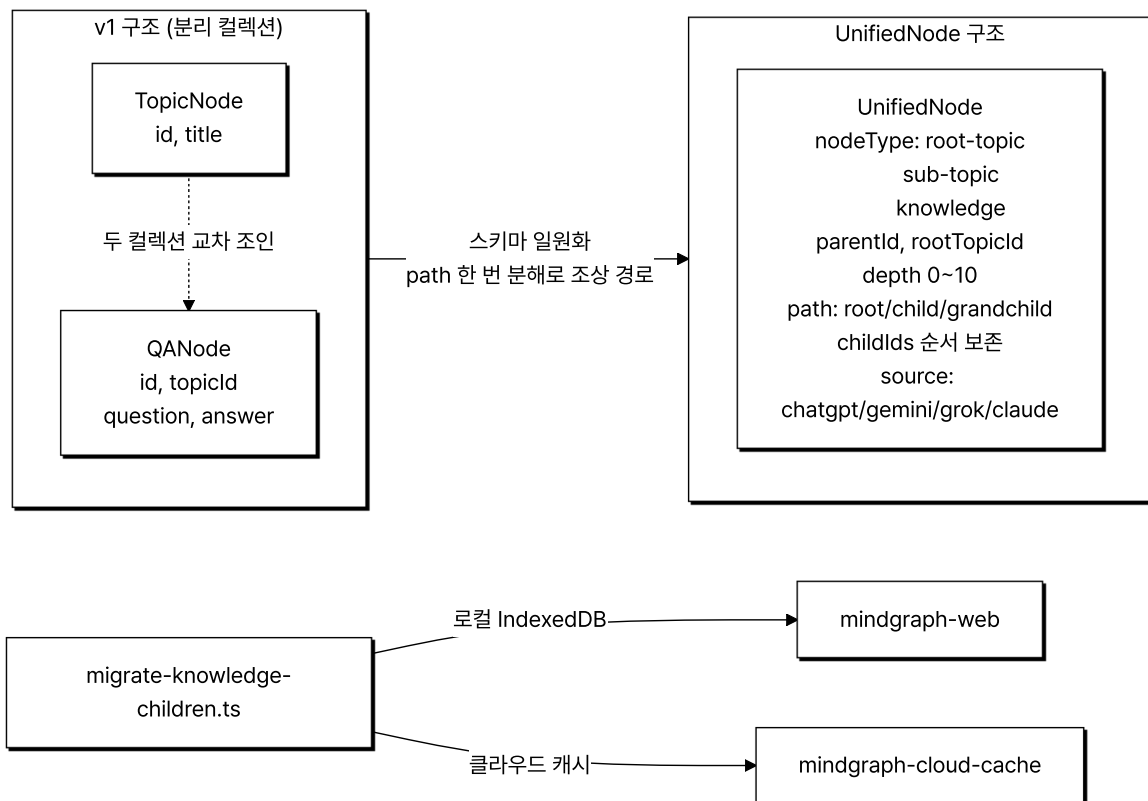
- **성과:** 사용자 행동(PostHog)·에러(Sentry)·로그(OpenTelemetry)·피드백(api/feedback) 4개 데이터 소스를 한 PR 단위로 묶어, 출시 후 sprint retro에서 다음 가설 입력으로 곧장 이어 붙일 수 있는 관측성 인프라를 사전 확보했습니다.
- **배운 점:** PostHog·Sentry·OpenTelemetry·api/feedback 4개 채널을 같은 PR 단위로 묶은 결과, 출시 후 받을 사용자 행동·에러·피드백이 한 retro 자리에서 다음 plan 입력으로 곧장 이어지는 동선이 코드 베이스에 사전 마련됐습니다.

Case 4. UnifiedNode 단일 스키마로 누적 데이터 모델 단순화

1. 문제 원인

- 초기 v1 구조에서 루트 토픽 **TopicNode** 와 질문·답변 **QANode** 를 별도 컬렉션으로 나눈 결정이, 코드 베이스에 신규 기능을 추가할 때마다 두 컬렉션 동기 비용을 다시 치르게 했습니다.
- 노드 조상 경로(breadcrumb) 조회에 두 컬렉션 재귀 조인이 필요해, PO가 새 가설(공유·검색·요약)을 코드로 옮길 때마다 같은 조인 로직을 재작성하는 비용이 누적되었습니다.
- 분리 컬렉션에서 단일 컬렉션으로 전환 시 무손실 마이그레이션 전략이 없으면 출시 후 사용자 데이터 손실로 직결될 위험이 있다고 판단했습니다.

2. 해결 과정



- **UnifiedNode 단일 컬렉션:** `nodeType` (`root-topic` · `sub-topic` · `knowledge`) 필드 하나로 모든 계층을 구분하고, `parentId` · `childIds` 로 부모-자식 관계를, `rootTopicId` 로 소속 루트를 추적했습니다.
- **path 필드 선도입:** "root-id/child-id/grandchild-id" 형태로 조상 경로를 노드에 기록해 컬렉션 조인 없이 path 한 번의 문자열 분해로 breadcrumb을 구성했습니다.
- **MAX_DEPTH = 10:** 무한 재귀 방지용 깊이 제한을 `schema.ts` 상수로 정의해 클라이언트 렌더링 범위를 한정했습니다.

- **무손실 마이그레이션:** `migrate-knowledge-children.ts` 가 로컬 IndexedDB(`mindgraph-web`)와 클라우드 캐시(`mindgraph-cloud-cache`) 양쪽을 동시 변환해, 로그인·비로그인 사용자의 기존 데이터를 UnifiedNode 스키마로 자동 전환했습니다.

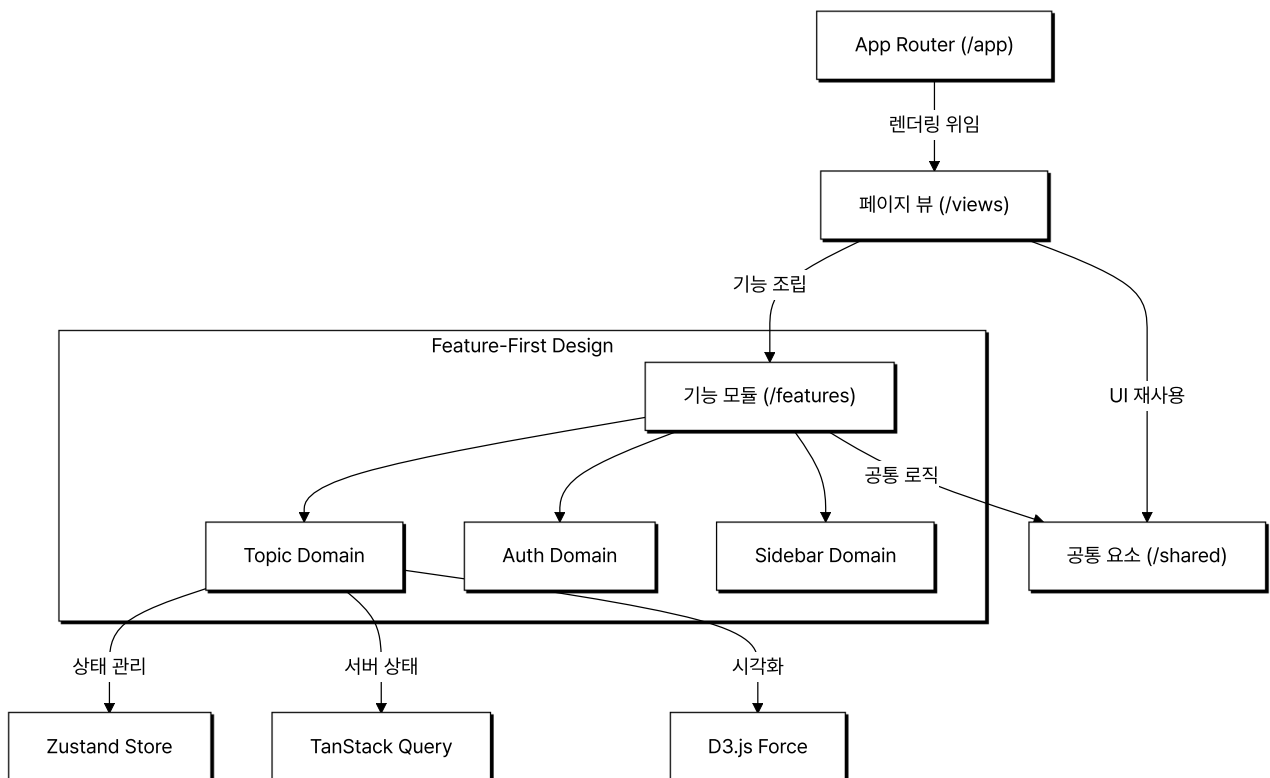
3. 결과

- **성과:** 단일 스키마로 10계층 트리를 표현하면서 조상 경로 조회를 path 한 번 분해로 처리했고, 기존 사용자 데이터는 앱 로드 시 자동으로 최신 스키마로 전환되어 데이터 유실 없는 업그레이드를 확보했습니다.
- **배운 점:** UnifiedNode로 합친 뒤 조상 경로 조회를 path 한 번 분해로 처리해, 신규 기능 추가 시 두 컬렉션 동기 코드를 재작성하지 않게 됐습니다.

[CHATGRAPH] - AI 대화 시각화 플랫폼

기존 선형 채팅 UI에서는 한 갈래 흐름만 살아남고 도중에 떠올린 분기 질문과 맥락이 휘발되어, 깊이 있게 사고를 확장하려는 사용자가 이전 맥락을 다시 짜맞춰야 하는 비용이 컸습니다. chatGraph는 비선형 사고 흐름을 보존하고 싶은 학습자와 리서처를 대상으로, AI와의 대화를 꼬리물기 형태의 그래프로 시각화하여 분기와 깊이를 잃지 않고 사고를 확장할 수 있도록 돕는 것을 목표로 했습니다. 본인은 4인 팀의 FE 한 명으로 참여해 응답 대기 시 사용자 흐름 단축과 시각화 라이프사이클 안정화를 담당했습니다.

전체적인 아키텍처



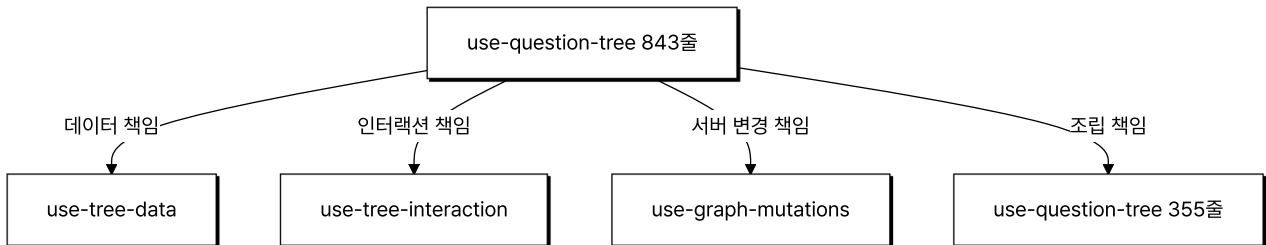
- Next.js App Router 위에 Feature-First 디렉토리 구조를 적용해 도메인 기능과 페이지 조립, 공통 자원을 분리했습니다.
- 본인은 FE 2명 중 1명으로 참여했으며, 아키텍처 재정비, 새 토픽 Optimistic 흐름, `useSuspenseQuery` 기반 데이터 패칭, 뷰 컴포지션 분리, 테스트 코드 작성을 담당했습니다.
- D3 기반 그래프 시각화의 색상 알고리즘과 호버 인터랙션은 팀원이 주도했고, 본인은 React 라이프사이클 통합과 cleanup 로직을 담당했습니다.

Case 1. Feature-First 디렉토리 도입과 거대 혹 분해

1. 문제 원인

- 초기 구조에서는 페이지, 기능, 공통 코드의 경계가 모호하여 컴포넌트끼리 서로를 직접 import하는 순환 참조가 반복 발생했습니다.
- 트리 상태와 인터랙션을 모두 끌어안은 use-question-tree 혹이 843줄까지 비대해져 변경 시 영향 범위 파악이 어려웠습니다.

2. 해결 과정



- app은 라우팅, features는 도메인, views는 조립, shared는 공통이라는 네 가지 책임으로 디렉토리를 재정의하고 import 방향을 단방향으로 정비했습니다.
- 단일 혹이 들고 있던 책임을 트리 데이터(use-tree-data), 인터랙션 상태(use-tree-interaction), 서버 변경(use-graph-mutations)으로 분리하고 최상위에서 합성하는 형태로 정리했습니다.
- 해당 작업은 본인 단독 커밋 3건에 걸친 점진적 리팩토링으로 진행했으며, 단일 거대 PR이 아닌 분해 방식으로 운영 중 위험을 분산했습니다.

3. 결과

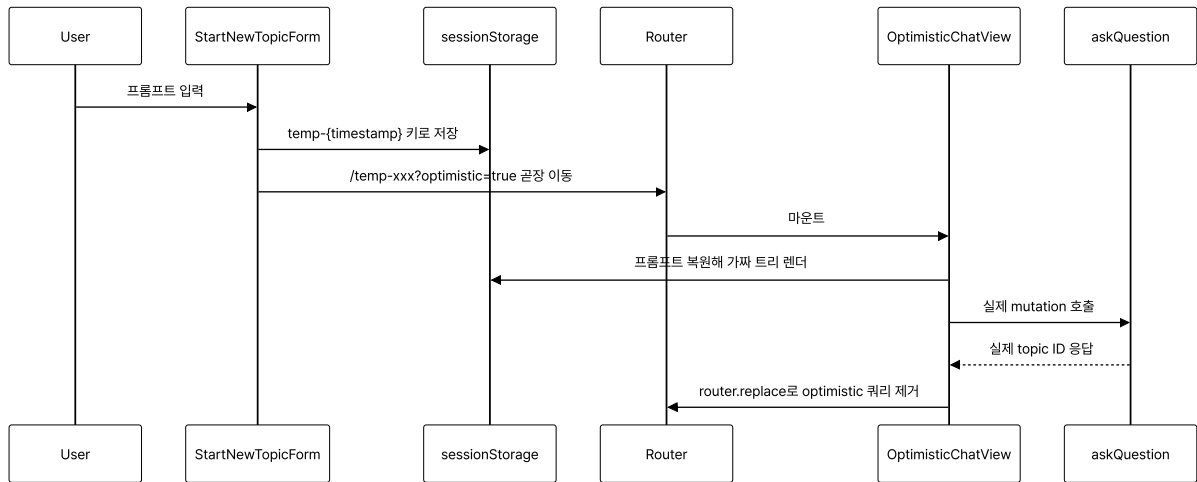
- 핵심 혹을 843줄에서 355줄로 축소하고 인접한 페이지 컴포넌트도 186줄에서 59줄로 정리하여 도메인별 책임을 명확히 했습니다.
- 기능 추가 시 어떤 디렉토리에 어떤 파일이 들어가야 하는지에 대한 팀 내 합의가 잡혀 신규 기능 머지 충돌 빈도가 줄었고, 다음 가설 검증을 받아낼 모듈 구조가 마련되었습니다.
- app-features-views-shared 네 책임으로 import 방향을 단방향으로 고정한 뒤 신규 기능 머지 충돌이 줄었습니다.

Case 2. 새 토픽 생성 시 LLM 응답 대기 가리기

1. 문제 원인

- 메인 화면에서 첫 질문을 입력하면 LLM 응답이 도착할 때까지 화면이 정지된 듯한 경험이 사용자 몰입을 끊었습니다.
- 단순히 토스트나 스피너를 띄우는 방식은 새 토픽이 생성될 URL을 미리 보여주지 못해 뒤로가기와 새로고침 동작이 어색했습니다.

2. 해결 과정



- StartNewTopicForm에서 temp-{Date.now()} 형태의 임시 ID를 만들어 sessionStorage에 프롬프트를 보관한 뒤 곧장 해당 경로로 push하도록 흐름을 설계했습니다(use-start-new-topic.ts).
- 이동한 경로에서는 useOptimisticTopicData 혹은 sessionStorage 데이터를 복원해 한 단계짜리 트리를 곧장 렌더링하고 동시에 실제 mutation을 실행합니다.
- mutation 성공 시 Zustand 스토어에 응답을 채워 두고 router.replace로 optimistic 쿼리 파라미터를 제거하여 실제 topic ID 경로로 자연스럽게 정착시켰습니다.

3. 결과

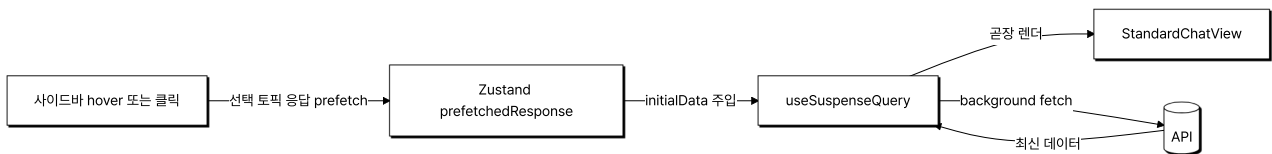
- LLM 응답까지의 대기 시간 동안에도 사용자가 자신이 입력한 질문이 그려진 화면을 곧장 볼 수 있어 체감 대기 시간이 줄었고, 첫 입력에서 첫 화면 사이의 사용자 흐름 단절을 없애 가장 큰 이탈 지점을 막았습니다.
- 임시 URL과 실제 URL의 전환을 사용자에게 노출하지 않도록 처리해 새로고침이나 공유 링크 동작도 정상적으로 유지했습니다.
- sessionStorage 임시 키·temp ID 라우팅·mutation 성공 시 router.replace를 한 흐름으로 묶어 첫 입력에서 첫 화면까지의 단절을 없앴습니다.

Case 3. 라우트 전환 시 빈 화면을 없애는 프리패치 패턴

1. 문제 원인

- 사이드바에서 다른 토픽을 클릭하면 useQuery가 새로 호출되며 페이지가 잠시 빈 상태로 보였습니다.
- App Router의 Suspense fallback이 노출되며 깜빡임이 발생했고, 데이터 양이 많은 토픽일수록 체감 지연이 커졌습니다.

2. 해결 과정



- 사이드바에서 토픽 응답을 미리 받아 Zustand 스토어의 prefetchedResponse 슬롯에 저장하도록 했습니다.
- 페이지 진입 시 useSuspenseQuery의 initialData에 동일 topicId의 프리패치 응답이 있으면 그대로 사용하고(use-topic-data.ts), 없을 때만 Suspense fallback을 거치도록 분기했으며, 다음 단계에서 백엔드 응답을 transformApiDataToViewData 어댑터로 트리 형태로 변환해 주입했습니다(use-tree-data.ts).
- 사용한 프리패치 데이터는 effect에서 정리해 다음 토픽 전환 시 잔여 데이터가 섞이지 않도록 했습니다.

3. 결과

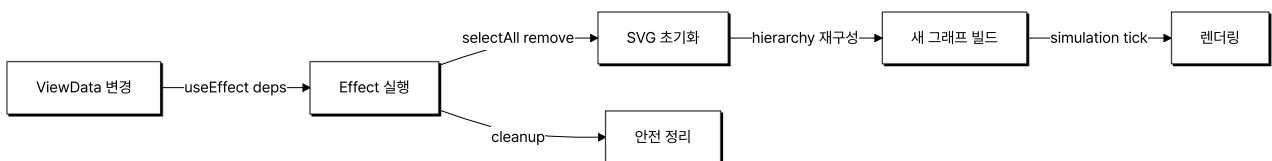
- 사이드바 기반 라우트 전환에서 빈 화면 노출 없이 트리가 지연 없이 그려지는 흐름을 확보해, 사용자가 토픽 사이를 빠르게 오가는 탐색 경험을 받아낼 수 있게 했습니다.
- useSuspenseQuery initialData와 Zustand prefetchedResponse 슬롯을 같은 topicId로 맞춰, 사이드바 토픽 전환에서 Suspense fallback이 노출되는 경우를 프리패치 미적재 경로로만 한정했습니다.

Case 4. D3 그래프와 React 라이프사이클의 안전한 통합

1. 문제 원인

- 팀원이 도입한 D3 Force Simulation은 SVG DOM을 직접 변형하기 때문에, React가 동일한 SVG를 다시 그리려고 할 때 잔여 노드나 이벤트 리스너가 남아 시각적 아티팩트가 발생했습니다.
- 토픽이 바뀌거나 트리가 갱신될 때마다 이전 시뮬레이션의 상태가 새 데이터 위에 덧씌워지는 문제가 있었습니다.

2. 해결 과정



- visualizer 컴포넌트의 useEffect에서 `d3.select(svgRef).selectAll("*").remove()` 로 매 갱신마다 SVG 하위 트리를 초기화한 뒤 hierarchy와 simulation을 다시 구성하는 패턴을 정리했습니다.
- onNodeClick과 같이 자주 변하는 콜백은 ref로 감싸 effect 의존성에서 분리하고, currentPath 등 시각적 강조에 영향을 주는 값만 deps에 포함해 불필요한 재구성을 막았습니다.
- 색상 알고리즘과 호버 desaturate 같은 시각적 인터랙션은 팀원이 작성한 코드를 유지하고, 본인은 통합 지점인 라이프사이클과 cleanup 흐름에 집중했습니다.

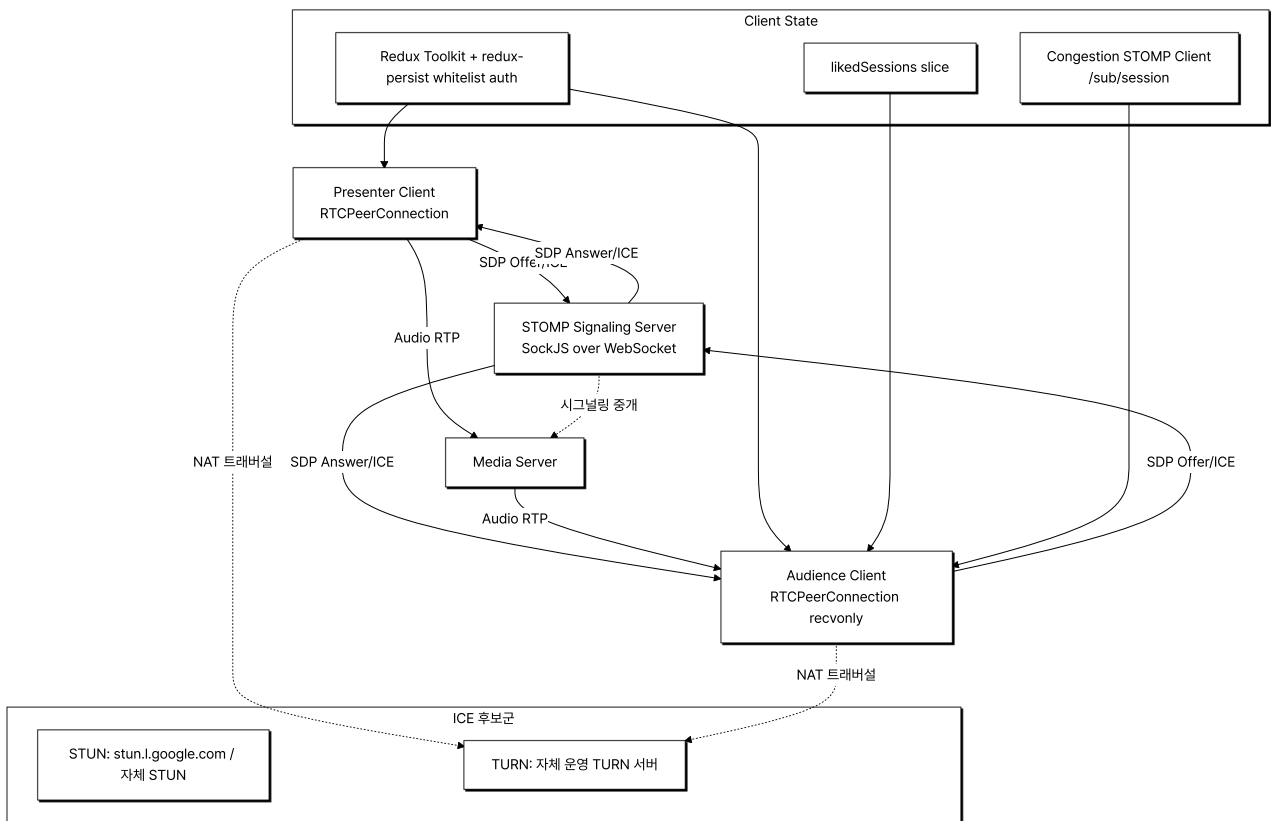
3. 결과

- 토픽이 바뀌거나 트리가 갱신될 때 SVG가 정상적으로 재구성되어 잔여 노드나 중복 이벤트로 인한 시각적 결함이 사라졌고, 사용자가 토픽을 오가도 시각 표현이 일관되게 유지되도록 안정화했습니다.
- 팀원이 작성한 D3 시각화 코드는 그대로 두고 통합 지점인 useEffect cleanup과 selectAll remove만 본인이 맡아, 토픽 전환 시 잔여 노드·중복 이벤트로 인한 시각적 결함을 없앴습니다.

[FIT] - WEBRTC 시그널링을 직접 구현한 사일런트 컨퍼런스 플랫폼

행사장에서 다중 채널 발표를 무선 헤드폰으로 골라 듣는 사일런트 컨퍼런스의 온라인·오프라인 하이브리드 중계 플랫폼입니다. 본인은 FE 3명 중 한 명으로 참여해 WebRTC 시그널링 클라이언트와 STOMP 채널 분리, 재연결 처리를 담당했습니다.

전체적인 아키텍처



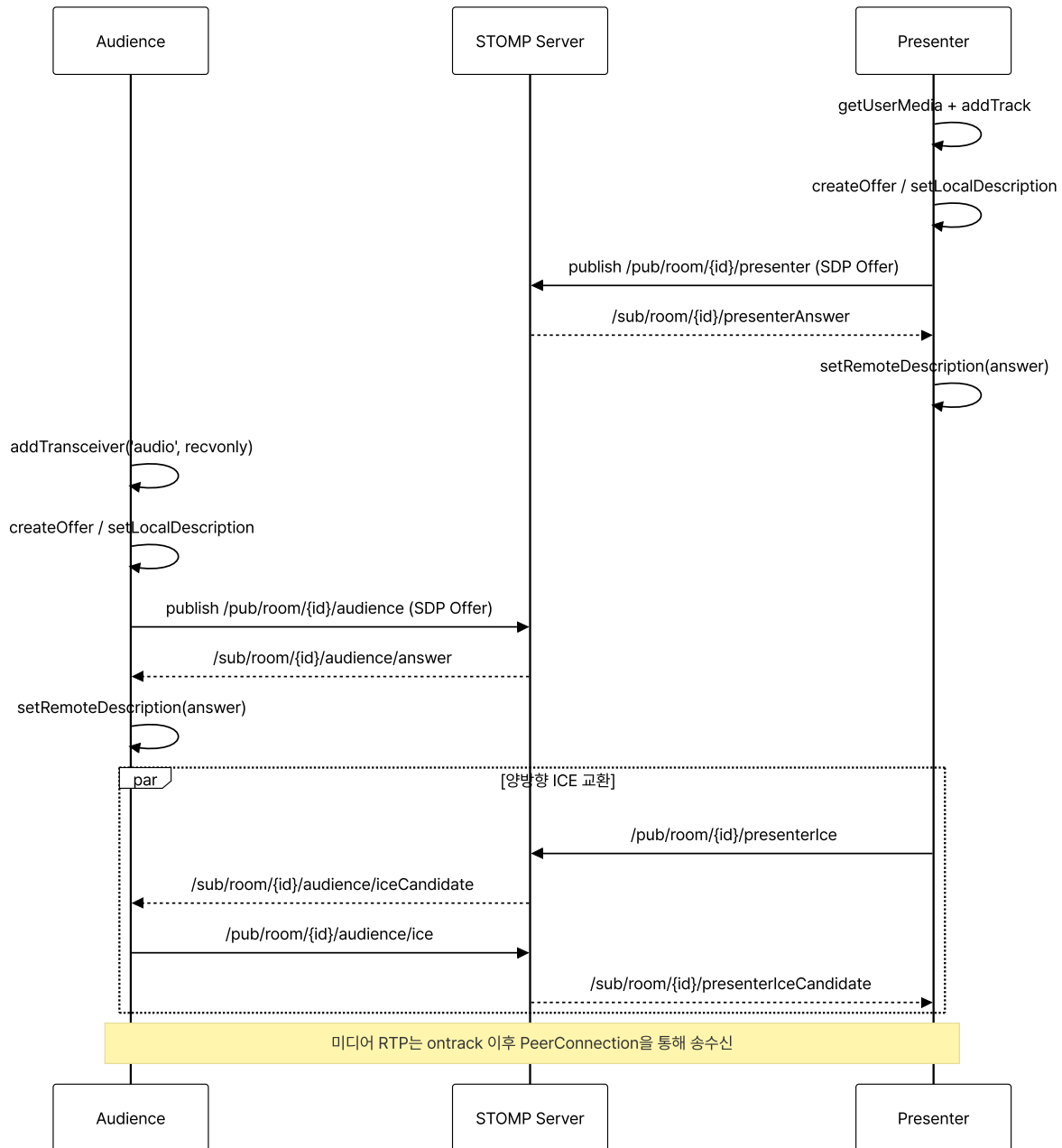
- **Architecture:** Vite와 React 환경에서 RTCPeerConnection API를 라이브러리 래퍼 없이 직접 사용하여, 발표자와 청중이 각각 미디어 서버와 1:N으로 연결되는 WebRTC 구조를 클라이언트 측에서 구현했습니다.
- 시그널링은 SockJS 위의 STOMP 프로토콜에 직접 정의했으며, 발표자는 `/pub/room/{id}/presenter` 로 청중은 `/pub/room/{id}/audience` 로 SDP 오퍼와 ICE를 publish하고 각자의 응답 토픽을 subscribe하여 양방향 협상을 완료합니다.
- 청중은 `pc.addTransceiver('audio', { direction: 'recvonly' })` 로 수신 전용 트랜시버를 명시하여 SDP 협상 비용을 최소화했고, NAT 환경 대응을 위해 자체 운영 TURN 서버를 ICE 후보군에 함께 등록했습니다.
- 인증 토큰과 좋아요 세션 목록은 Redux Toolkit으로 관리하고 redux-persist whitelist에 auth만 포함하여 보안과 연속성을 함께 확보했습니다.

Case 1. STOMP 위에 WebRTC 시그널링을 직접 구현하여 저지연 오디오 송수신을 구성

1. 문제 원인

- 사일런트 컨퍼런스는 같은 공간에서 무선 헤드폰으로 발표를 청취하는 형태라, 발표자의 입모양과 오디오 사이의 싱크가 어긋나면 체감 경험이 무너집니다.
- HLS/RTMP 계열의 라이브 스트리밍은 수 초 이상의 버퍼링 지연이 발생하기 때문에 현장 청취 용도로 부적합했고, 수백 밀리초 단위의 RTT를 확보하는 WebRTC 기반 구성이 필요했습니다.
- 사내 인프라에는 별도의 WebRTC SDK가 없었고, 시그널링 채널을 새로 정의하면서 발표자와 청중 양측의 PeerConnection 라이프사이클을 클라이언트에서 직접 관리해야 했습니다.

2. 해결 과정



- 발표자 클라이언트에서 **getUserMedia** 로 마이크 스트림을 받아 **addTrack** 으로 PeerConnection에 부착하고, **createOffer / setLocalDescription** 으로 만든 SDP를 STOMP **/pub/room/{id}/presenter** 토픽에 publish하도록 구성했습니다.
- 청중 클라이언트는 음성 송출이 필요하지 않으므로 **pc.addTransceiver('audio', { direction: 'recvonly' })** 로 수신 전용 트랜시버를 추가한 뒤 별도 PeerConnection을 만들어 **/pub/room/{id}/audience** 로 오퍼를 보냈습니다.
- **onicecandidate** 콜백에서 생성되는 ICE 후보는 발표자/청중 각각 **/pub/room/{id}/presenterIce** , **/pub/room/{id}/audience/ice** 토픽으로 양방향 publish하여 서버 중개 시그널링을 완성했습니다.
- 자체 운영 중인 TURN 서버와 자체 STUN, 공용 STUN을 **iceServers** 배열에 함께 등록하여 행사장 NAT 환경에서도 연결률이 떨어지지 않도록 했습니다.

3. 결과

- **성과**: 발표자와 청중이 동일한 STOMP 채널 위에서 SDP와 ICE를 교환하는 1:N 오디오 스트리밍 구성을 완성했으며, RTCPeerConnection의 RTP 전송 특성상 HLS 대비 체감 지연이 크게 줄어 현장 청취 경험을 확보했습니다.

- **배운 점:** 라이브러리 래퍼 없이 createOffer.setLocalDescription.addIceCandidate를 직접 호출하면서 SDP-ICE 도착 순서를 컴포넌트 상태로 관리하는 흐름을 정리했고, STOMP 시그널링 트래픽과 RTP 미디어 경로를 분리해 처리하도록 코드를 구성했습니다.

Case 2. SDP 협상 이전에 도착한 ICE 후보를 큐잉하여 연결 안정성 확보

1. 문제 원인

- 시그널링 초기 구현에서는 SDP 응답을 받기 전에 ICE 후보가 먼저 도착하면 `setRemoteDescription` 이 호출되지 않은 `PeerConnection`에 `addIceCandidate` 를 시도하면서 예외가 발생했습니다.
- 후보가 한 번이라도 유실되면 NAT 트래버설이 실패하여 미디어 트랙이 붙지 않는 무음 상태가 재현되었고, 실패 시 별다른 복구 경로가 없어 사용자가 페이지를 새로고침해야 했습니다.
- 원인은 STOMP `presenterAnswer / audience/answer` 메시지와 `presenterIceCandidate / audience/iceCandidate` 메시지의 도착 순서가 정렬되지 않는다는 점에 있었습니다.

2. 해결 과정

- 발표자와 청중 컴포넌트 양쪽에 `pendingCandidates` 라는 `useRef` 큐를 두고, ICE 메시지를 수신했을 때 `pcRef.current.remoteDescription` 이 아직 비어 있으면 곧장 추가하지 않고 큐에 적재하도록 했습니다.
- SDP Answer 수신 시점의 `setRemoteDescription` 콜백 안에서 큐에 쌓인 후보를 순서대로 `addIceCandidate` 로 flush하고 큐를 비우는 흐름을 구성하여, 메시지 도착 순서와 무관하게 후보가 유실되지 않도록 했습니다.
- 큐를 컴포넌트 로컬 ref로 둔 이유는 `PeerConnection` 자체가 컴포넌트 단위의 라이프사이클을 가지기 때문이며, 전역 스토어에 두면 라우팅 전환이나 언마운트 시 잔여 후보가 다음 세션으로 누수될 위험이 있었습니다.

3. 결과

- **성과:** SDP와 ICE 도착 순서 역전으로 인한 초기 연결 실패가 사라졌고, 발표자/청중 양측의 `PeerConnection`이 ICE Connected 상태에 도달하는 동작이 일관되었습니다.
- **배운 점:** `useRef` 큐로 ICE 후보를 적재하고 `setRemoteDescription` 직후 flush한 뒤 `PeerConnection`이 ICE Connected 상태에 일관되게 도달했습니다.

Case 3. 실시간 채팅·혼잡도 채널의 STOMP 재연결과 heartbeat 설정

1. 문제 원인

- 행사장의 Wi-Fi와 모바일 망은 일시적으로 끊기는 일이 잦았고, 실시간 채팅과 혼잡도 표시를 나르는 STOMP 세션이 재연결되지 않으면 청중이 채팅과 세션별 인원 변동을 받지 못하는 현상이 발생했습니다.
- 기본 STOMP 설정 상태에서는 WebSocket이 끊어진 뒤 클라이언트가 알아채는 시점이 늦어 좀비 커넥션이 유지되었고, 끊김을 감지하더라도 수동으로 재연결을 트리거해야 해서 사용자 입장에서는 페이지를 새로고침하지 않으면 복구되지 않았습니다.

2. 해결 과정

- 채팅 STOMP 클라이언트(`api/chat/stomp-client.js`)와 혼잡도 STOMP 클라이언트(`api/congestion/congestion-stomp-client.js`)에 각각 `reconnectDelay: 5000` 을 지정하여 연결이 끊긴 뒤 5초 후 자동으로 재연결 시도가 이뤄지도록 했고, 새 세션에서도 등록해 둔 토픽 구독이 자동으로 복원되도록 구독 캐시를 함께 두었습니다.
- 두 클라이언트에 `heartbeatIncoming: 4000` 과 `heartbeatOutgoing: 4000` 을 함께 설정하여 4초 간격의 양방향 heartbeat로 좀비 커넥션을 빠르게 감지하고, 네트워크 단절을 STOMP 레벨에서 명시적으로 인지하도록 했습니다.
- 재연결 이후에도 권한이 유지되도록 인증 토큰을 `connectHeaders` 와 각 `subscribe` 호출 헤더에 모두 `Bearer` 형식으로 실어 보내, 백엔드 인터셉터의 토큰 검증 흐름과 정합성을 맞췄습니다.

3. 결과

- **성과:** 일시적 네트워크 변동에서도 채팅·혼잡도 STOMP 세션이 자동으로 복구되어, 사용자가 페이지를 새로고침하지 않아도 채팅 메시지와 세션별 인원 변동이 끊기지 않고 이어지는 동작을 확보했습니다.

- **배운 점:** reconnectDelay 5초와 4초 heartbeat를 채팅·혼잡도 클라이언트에 동일하게 적용한 뒤 단절된 세션이 새로고침 없이 복구됐습니다.

Case 4. 혼잡도 데이터를 독립 세션으로 분리한 STOMP 채널 이중화

1. 문제 원인

- 사일런트 컨퍼런스는 다수의 채널이 병렬로 진행되기 때문에 청중이 어떤 세션이 붐비는지 실시간으로 파악하지 못하면 채널 이동 의사결정을 내리기 어려웠고, 혼잡도 브로드캐스트가 채팅 메시지 흐름과 분리되어 있어야 운영 화면이 끊김 없이 갱신될 수 있었습니다.
- 그러나 채팅 STOMP 클라이언트 하나에 혼잡도 토픽을 함께 묶으면 채팅 메시지가 몰리는 구간에서 혼잡도 수신이 밀리거나, 반대로 혼잡도 브로드캐스트가 채팅 채널을 점유할 위험이 있었습니다.

2. 해결 과정

- 혼잡도 전용 STOMP 클라이언트(`api/congestion/congestion-stomp-client.js`)를 채팅 클라이언트와 별도로 구성하여 같은 SockJS 엔드포인트 위에 독립된 STOMP 세션을 띄우고, `/sub/session` 토픽 하나만 구독하도록 책임 범위를 좁혔습니다.
- 채팅 STOMP 클라이언트와 동일한 `reconnectDelay: 5000` , `heartbeatIncoming/Outgoing: 4000` 파라미터를 적용하여 두 채널이 같은 회복 정책을 가지면서도 서로의 트래픽 폭증에 영향을 받지 않도록 격리했습니다.
- 채널을 둘로 나누면서 메시지 도메인 자체를 분리할 수 있게 되어, 혼잡도 데이터는 화면 갱신 콜백으로 바로 흘러보내고 채팅 메시지는 채팅 렌더링 로직에 집중하도록 클라이언트 측 책임을 명확히 했습니다.

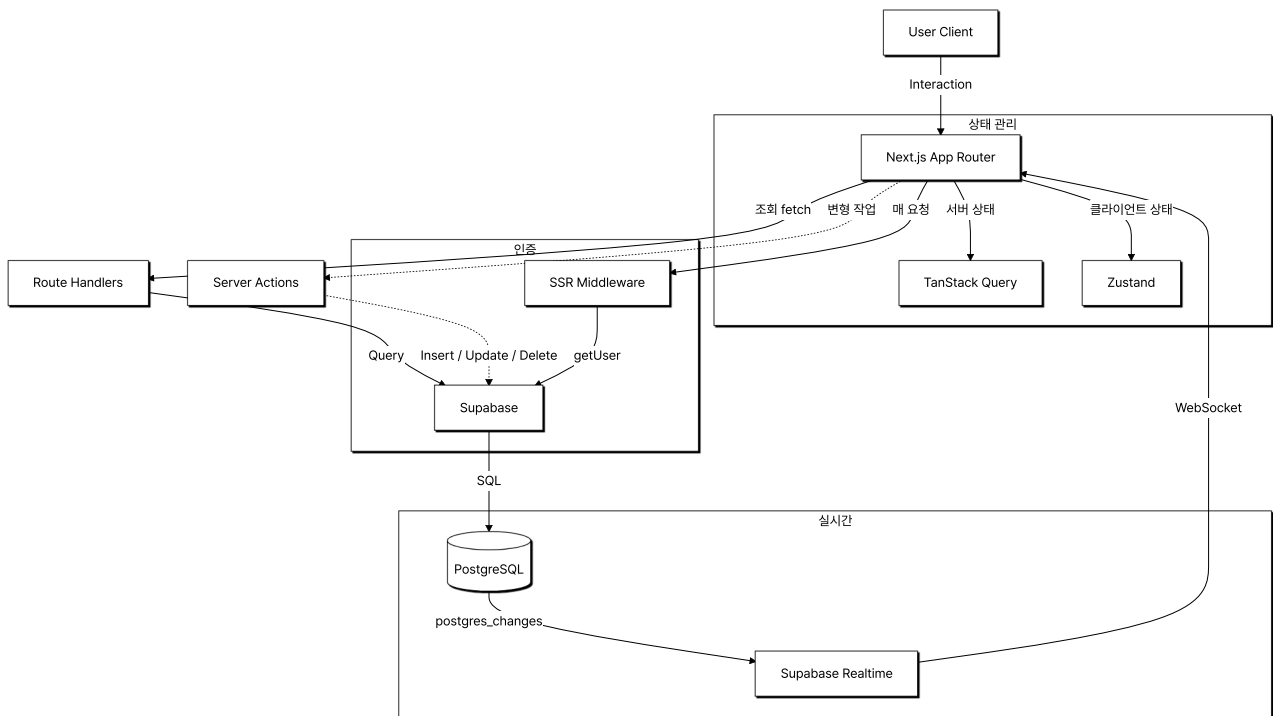
3. 결과

- **성과:** 채팅과 혼잡도가 서로 다른 STOMP 세션을 통해 흐르게 되어 한쪽 트래픽이 폭증해도 다른 쪽이 영향을 받지 않는 구조가 마련되었고, 청중 화면에 채널별 인원 변동을 별도 라인으로 실시간 반영할 수 있게 되었습니다.
- **배운 점:** 혼잡도 전용 STOMP 클라이언트를 분리한 뒤 채팅 트래픽이 몰리는 구간에서도 혼잡도 화면 갱신이 밀리지 않았습니다.

[XAB] - 실시간 투표 및 소통 중심의 A/B 테스트 SNS

두 선택지에 대한 실시간 투표·댓글 토론이 공장 반영되는 SNS로, 본인은 4인 FE 팀에 참여해 Next.js App Router 위에서 모달 라우팅·실시간 동기·미들웨어 인증·OAuth 콜백 오리진 복원을 담당했습니다.

전체적인 아키텍처



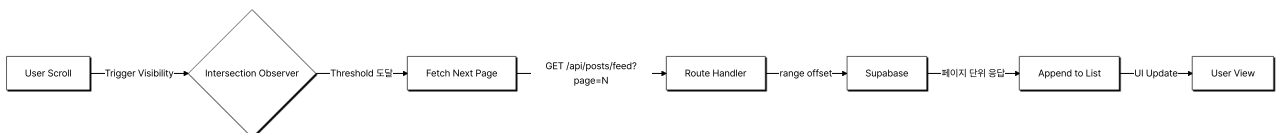
- Next.js App Router 기반 프론트엔드와 Supabase 백엔드를 결합하여 구축했습니다.
- 조회성 요청은 클라이언트 컴포넌트가 Route Handler를 호출해 Supabase에 접근하는 경로로 처리하고, 게시물 작성·수정·삭제 같은 변형 작업은 Server Actions가 Supabase를 직접 호출하도록 두 경로를 명확히 분리했습니다.
- 서버 상태는 TanStack Query, 클라이언트 전역 상태는 Zustand로 책임을 분리하여 실시간성이 중요한 SNS 환경에서도 캐시 일관성과 사용자 인터랙션 상태를 한 흐름에서 관리했습니다.

Case 1. 피드 초기 로딩 지연을 해소하는 지연 로딩 전략

1. 문제 원인

- 메인 피드 접속 시 게시물, 투표 옵션, 댓글 수, 좋아요 정보를 한 번에 불러오는 구조에서는 첫 화면 진입 시 응답 페이로드와 렌더링 비용이 크게 누적되었습니다.
- 뷰포트 밖에 있는 게시물까지 동일한 비중으로 페칭과 렌더링을 수행하면서 초기 화면이 그려지기까지의 체감 지연이 발생했습니다.

2. 해결 과정



- 사용자가 당장 보지 않는 데이터까지 한 번에 호출하는 대신 피드를 페이지 단위로 분할하여 호출하는 전략을 수립했습니다.
- **useInfiniteQuery** 로 페이지네이션 상태를 관리하고, **react-intersection-observer** 로 뷰포트 하단 트리거 요소의 노출 시점을 감지해 다음 페이지를 가져오도록 구현했습니다.
- Route Handler는 **page** 쿼리 파라미터를 받아 Supabase의 **.range(offset, offset + limit - 1)** 메서드로 페이지 단위 응답만 반환하도록 구성하여 서버와 클라이언트가 동일한 분할 단위를 공유하게 했습니다.

3. 결과

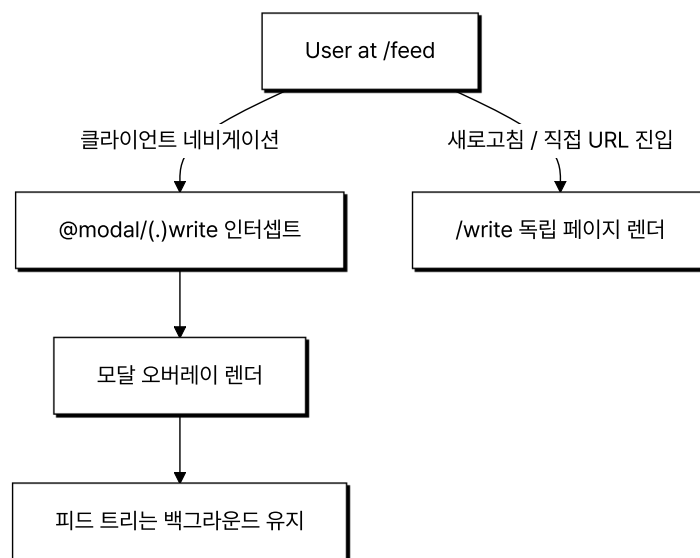
- 초기 진입 시 필요한 데이터만 우선 응답하도록 변경하여 첫 화면이 그려지기까지의 체감 지연을 완화했습니다.
- 스크롤 위치에 따라 점진적으로 데이터를 채워 사용자의 탐색 흐름이 끊기지 않는 무한 스크롤 경험을 제공했습니다.
- `useInfiniteQuery`와 `IntersectionObserver`로 페이지 단위 분할 호출을 적용해 첫 화면 응답 페이로드와 렌더링 비용을 줄였습니다.

Case 2. 피드 맥락을 유지하는 모달 라우팅 패턴 적용

1. 문제 원인

- 피드에서 게시물 작성이나 상세 화면으로 이동할 때 전체 페이지가 다시 렌더되면서 스크롤 위치와 진행 중이던 인터랙션이 모두 초기화 되는 문제가 있었습니다.
- 기존 라우팅 방식은 페이지 컨텍스트를 완전히 교체하기 때문에 이전 화면의 상태를 유지한 채 상호작용할 수 없는 구조적 한계가 원인이었습니다.

2. 해결 과정



- 탐색 흐름을 유지하기 위해 피드 트리는 그대로 두고 그 위에 모달을 띄우는 방식을 채택했습니다.
- Next.js App Router의 병렬 라우트 슬롯(`@modal`)과 인터셉팅 라우트(`(.)write`)를 함께 배치하여, 클라이언트 네비게이션 시에는 모달 오버레이가 렌더되고 직접 URL 진입이나 새로고침 시에는 독립 페이지가 렌더되는 구조를 만들었습니다.
- `@modal/default.tsx` 에 빈 슬롯 폴백을 두어 슬롯이 활성화되지 않은 라우트에서도 트리가 흐트러지지 않도록 처리했습니다.

3. 결과

- 게시물 작성과 상세 진입 시 피드 위치와 스크롤 컨텍스트가 보존되어 탐색이 끊기지 않는 경험을 제공했습니다.
- 모달 진입과 독립 페이지 진입을 같은 URL로 처리할 수 있어 공유 링크나 새로고침 시에도 일관된 화면을 유지했습니다.
- `@modal` 병렬 라우트와 `(.)write` 인터셉팅 라우트를 결합해 클라이언트 네비게이션은 모달, 직접 URL 진입은 독립 페이지로 갈라지는 흐름을 구성했습니다.

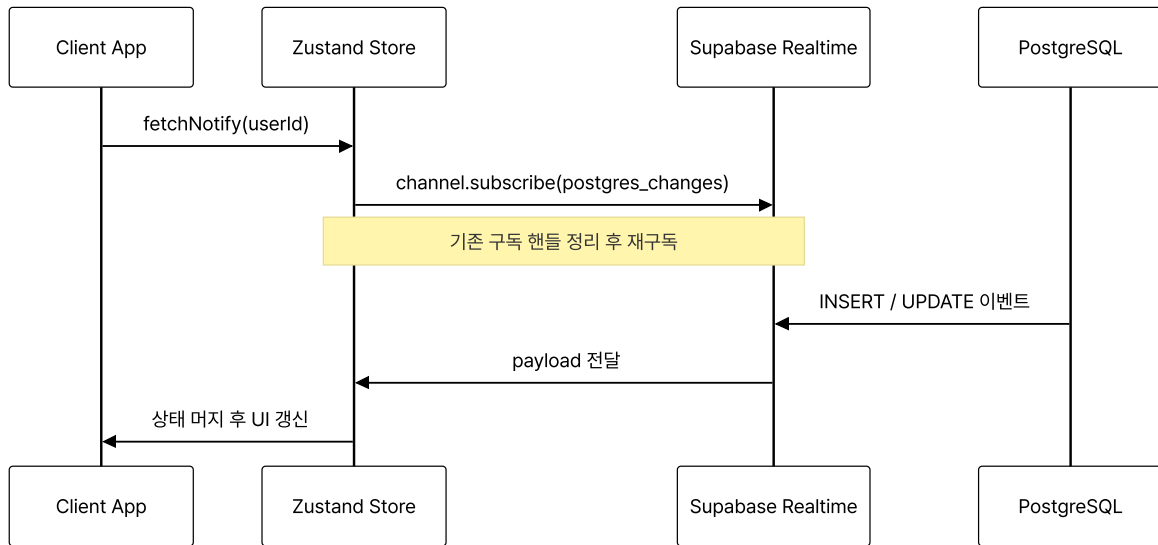
Case 3. Supabase Realtime을 활용한 라이브 피드 환경 구축

1. 문제 원인

- SNS 특성상 댓글과 투표가 빈번하게 발생하지만, 사용자가 직접 새로고침을 누르기 전까지는 최신 데이터를 확인할 수 없는 정적인 환경이 소통을 저해했습니다.

- 주기적인 폴링 방식은 변화가 없을 때도 호출이 반복되어 서버 부하와 네트워크 비용을 누적시키는 구조적 비효율이 있었습니다.

2. 해결 과정



- 데이터베이스 변경 사항을 클라이언트로 곧장 푸시하기 위해 Supabase Realtime의 **postgres_changes** 이벤트 구독을 도입했습니다.
- 댓글 영역에서는 React Query 캐시 갱신 로직과 연동하여 새 댓글이 도착하면 캐시가 갱신되고 화면이 곧장 반영되도록 구성했습니다.
- 알림 영역에서는 Zustand 스토어가 구독 해제 핸들을 함께 보관하도록 설계하고, 재진입이나 사용자 전환 시 이전 구독을 정리한 뒤 재구독하도록 하여 중복 구독으로 인한 리소스 누수를 방지했습니다.

3. 결과

- 새로고침 없이도 새 댓글과 알림이 곧장 반영되는 라이브 환경을 구현하여 상호작용 응답성을 확보했습니다.
- 폴링 방식을 제거하고 변경 이벤트가 발생할 때만 메시지를 받도록 하여 불필요한 요청을 줄였습니다.
- Zustand 스토어가 구독 해제 핸들을 함께 보관하도록 두어 사용자 전환 시 이전 구독을 정리한 뒤 재구독해 중복 구독 누수를 막았습니다.